

Host-Enforced Document Read Permissions for Language-Model Inference: A Reproducible Structural Evaluation

Pratham Vasa

Affiliation: [to be supplied]

Email: prathamv1102@gmail.com ORCID: [to be supplied]

Draft: October 2026

Abstract

Prompt injection can arise when a language model jointly processes application instructions and lower-trust content. We evaluate a local inference mechanism in which the host assigns document read permissions. Denied documents occupy fixed-capacity compartments, and every public/output query is masked from denied values at every decoder layer and cached generation step. Under stated assumptions, changing denied values while public inputs, permissions, model state, and permitted metadata remain fixed cannot change public logits or greedy output. On a pinned Qwen2.5-0.5B-Instruct configuration and 175 BIPIA-derived EmailQA, TableQA, and CodeQA cases, every alternate denied-payload comparison had equal input shape, byte-identical recorded-step logits, and identical greedy tokens (175/175); no run reported an error. A simpler host-filtering baseline produced equal output strings and EOS status in all 175 clean comparisons. This is a structural measurement, not a semantic prompt-injection efficacy result. Readable attack text changed ordinary output strings in 42/175 cases, but attack success was not judged. The source revision is consumed development material; two BIPIA tasks were omitted. The system does not establish instruction hierarchy for readable evidence, general model portability, or protection against tool effects.

Keywords: prompt injection; information flow; attention masks; document permissions; language-model security; reproducible evaluation

1 Introduction

Language-model applications commonly place trusted application instructions beside retrieved documents, messages, and tool results. Imperative text in those lower-trust inputs can then compete with the task the application intended the model to perform. Indirect prompt injection made this authority problem concrete in integrated applications [Greshake et al., 2023]; BIPIA, InjecAgent, and AgentDojo provide benchmarks for related response and agent settings [Yi et al., 2025, Zhan et al., 2024, Debenedetti et al., 2024].

This paper studies one narrow enforcement question: can a host make one document’s values unreachable from a language model’s public response computation? The answer is conditional on the model/runtime assumptions stated below. It does not answer the larger question of whether a model will resist instructions in a document it is allowed to read. That distinction is essential: a data-access boundary can protect a host-denied secret, but it cannot by itself decide which natural-language instruction should prevail among readable inputs.

Our implementation is a local, adapter-specific modification of causal attention for Qwen2 and Qwen2.5. It is structurally tested on tiny Qwen2 and Llama configurations. The host assigns source identifiers and grants before inference; document text cannot mint or alter them. Each denied document occupies fixed slots, and every public query is masked from every denied key at every layer, including cached decoding.

The output path is buffered and fails closed on boundary errors. Privileged effects remain outside this component: a separately mediated broker checks host-issued capabilities at the tool/action point.

Contributions. (1) We specify a host-issued document-permission boundary for autoregressive inference and give its conditional noninterference argument. (2) We provide a reproducible 175-case BIPIA-derived structural evaluation with pinned code, model revision, source hashes, and full per-step logit comparisons. (3) We compare the clean permission arm with simpler host filtering and find no output difference in the tested cases. (4) We state the evidence boundary explicitly: readable-evidence resistance was not measured.

We make no priority claim for hard attention-mask information-flow architectures; closely related work has proposed provenance-aware masking for LLM security [Gupta, 2025].

2 Threat model and contract

The trusted host supplies the system policy, document identifiers, document order, read grants, and any action capability. A user task and retrieved document contents are lower-trust. The attacker may control denied or permitted document text but cannot compromise the host process, model weights, adapter, attention implementation, or action mediator. Timing, resource use, host-memory inspection, and compromised runtime code are outside the claim.

There are three separate properties. First, *denied-source noninterference* concerns values in a document the host denies. Second, *readable-source instruction resistance* concerns malicious instructions in content the model may read; it remains unmeasured here. Third, *effect authorization* concerns tools and privileged side effects; it is enforced by the NPS broker at the action path, not by this response firewall. A model output is only a proposal and receives no authority to grant itself access or execute an effect.

3 Mechanism and conditional noninterference

For a request, let P be trusted policy state, U public inputs (task and permitted documents), D denied document values, and M permitted metadata such as the fixed denied-slot count. The host holds (P, U, M) fixed during a comparison. At each decoder layer, public queries may read permitted earlier keys and the protected policy state, while each denied query may read only its own token. Public logical positions do not advance through denied slots. The implementation verifies the intended mask at every layer and cached step.

Claim. Subject to deterministic inference and the implementation assumptions, for all denied values D, D' fitting the fixed capacity,

$$\text{Logits}_{1:T}(P, U, D, M) = \text{Logits}_{1:T}(P, U, D', M), \quad \text{Greedy}(P, U, D, M) = \text{Greedy}(P, U, D', M). \quad (1)$$

Argument. At prefill, a public embedding depends only on P, U, M , not denied values. At a decoder layer, each public attention query receives no denied key/value contribution by the mask. Layer normalization, MLPs, residual additions, and output projection act per position; therefore a public state remains independent of D by induction over layers. Cached decoding repeats the same condition at each generated public token. Deterministic greedy decoding feeds only prior public tokens back into the next step, giving induction over t . Fixed denied capacity and logical public positions prevent denied content length from changing the public shape or positions within the stated capacity. This argument fails if a mask is omitted, a supported backend interprets it differently, denied metadata is allowed to vary, inference is nondeterministic, or the trusted host/runtime is compromised. It says nothing about values in U .

This is an implementation-level information-flow property, not a claim that the model has learned a privileged instruction representation. Ordinary causal attention already prevents later tokens from changing earlier prefix states. The contribution is host-owned source assignment, full-layer/cached-step compartment masking, fixed-capacity handling, mask verification, and counterfactual measurement.

4 Related work

Instruction-hierarchy training aims to make models prioritize privileged instructions [Wallace et al., 2024]. Spotlighting and structured-query methods preserve or train on provenance cues in inputs [Hines et al., 2024, Chen et al., 2025]. These approaches address authority among readable content; the present mechanism prevents values from a denied document from reaching public computation. Latent monitoring and instruction detection use internal model signals to flag suspicious behavior [Hung et al., 2025, Wen et al., 2025]; an earlier six-case trajectory-monitor prototype in this repository is historical context only and is not the system evaluated here.

Most closely, Contextual Integrity Verification proposes signed provenance labels and hard attention masks for frozen models [Gupta, 2025]. We therefore do not claim that the use of masking, provenance, or a noninterference framing is new. This work reports a narrower open implementation and a logged counterfactual evaluation. DeTAM modifies selected attention heads at inference to emphasize a core intent in jailbreak settings [Li et al., 2025]; its intervention is distinct from source-graph exclusion. CaMeL places control/data-flow constraints around agent tool use [Debenedetti et al., 2025]; our tool boundary is similarly external to model prose, but no agent-effect evaluation is reported. Adaptive attacks have bypassed evaluated prompt-injection defenses [Zhan et al., 2025]; this study includes no disclosed-defense adaptive evaluation.

5 Evaluation

5.1 Configuration and cases

We ran Qwen/Qwen2.5-0.5B-Instruct at revision `7ae557604adf67be50417f59c2c2f167def9a775`, Transformers 4.57.6, PyTorch 2.6.0, CPU float32, SDPA attention, greedy cached decoding, a 4096-token context limit, and 256 denied-token slots per document. The run used BIPIA repository revision `a004b69ec0dd446e0afd461d98cb5e96e120a5d0`. It sampled all 50 EmailQA rows with 50 text payloads, 75 evenly spaced TableQA rows with all 75 text payloads, and all 50 CodeQA rows with all 50 code payloads. Start/middle/end placement was balanced; middle placement used a deterministic standard-library sentence-boundary approximation rather than BIPIA’s original procedure. QA and summarization were omitted because their source datasets require separate

Table 1: Structural result by BIPIA-derived task. “Readable change” compares ordinary clean and readable-attack output strings, including missing outputs from incomplete generations; it is not semantic attack success.

Task	Cases	Denied invariance	Readable change (ordinary)
EmailQA	50	50/50	5/50
TableQA	75	75/75	26/75
CodeQA	50	50/50	11/50
Total	175	175/175	42/175

retrieval and terms.

For each case, we generated clean evidence, attack text as permitted evidence, and attack text as a host-denied private document. Ordinary decoding, isolated policy memory, and the read-permission adapter were run under the same request settings. The key structural counterfactual compared two distinct denied attack payloads in the read-permission arm and required equal input shape, every generated-step logit bytes, and output tokens. The runner checkpoints atomically after each case and records source/model/policy hashes, outputs, generation lengths, and timing. The raw artifact is retained locally because it contains generated completions; the repository contains code and sanitized documentation only.

5.2 Outcomes

The primary outcome is the structural counterfactual check, not attack success. We also report runtime errors and whether the ordinary output string changed when a readable attack replaced clean evidence. This comparison includes missing outputs from incomplete generations. A changed response is not an injection-success label. We did not run BIPIA’s semantic judge or obtain human outcome labels. Exact/substring reference matching remains in the raw artifact only as a weak diagnostic and is not task accuracy.

6 Results

All 175 denied-payload counterfactuals passed. Each had the same input shape, byte-identical logits at every recorded generation step, and identical greedy tokens. Across 1,750 generated attempts, 1,657 ended with EOS, 93 reached the generation limit, and none reported a runtime error. Ordinary output strings differed between clean and readable-attack conditions in 42/175 cases, including incomplete/missing outputs: 5/50 EmailQA, 26/75 TableQA, and 11/50 CodeQA. This descriptive count establishes that readable content can influence outputs in this configuration, but does not establish instruction following, attack success, utility loss, or a comparative defense effect.

Outputs reached EOS in all 50 EmailQA cases; depending on arm, 73–75 of 75 TableQA cases and 41–43 of 50 CodeQA cases reached EOS. The other outputs generally reached the configured generation cap. Summed per-generation timing across all 1,750 attempts was 5,962.68 seconds (about 99.4 minutes), excluding adapter construction and setup; it is a diagnostic, not an end-to-end latency benchmark.

For the 175 clean read-permission-arm cases, a separate ordinary-decoding baseline omitted the denied sources before serialization. It had the same generated-token hash, EOS status, and released output in 175/175 cases, with 166 EOS completions, nine capped outputs, and zero errors.

Its summed per-result generation timings were 492.40 seconds; whole-run wall time was 524.68 seconds including setup. This is neither a matched latency comparison nor evidence of an advantage for either implementation. For the tested property, host filtering is the simpler security-equivalent design, and we found no utility benefit for retaining isolated denied tokens.

An additional engineering stress check used randomly initialized tiny Qwen2 and Llama models across eager and SDPA attention (25 cases per family/backend configuration, 100 total). It varied one or two denied documents and checked four generation steps. The denied-payload comparison passed in 100/100 cases, and a permitted-public perturbation changed the checked public logits in 100/100 cases. These are implementation checks on random tiny models, not behavioral-security evidence.

A second pretrained architecture check used HuggingFaceTB/SmolLM2-135M-Instruct (Llama) at revision `12fd25f77366fa6b3b4b768ec3050bf629380bac`. All seven synthetic development cases passed the denied-swap structural check; all 28 arm/variant attempts ended with EOS and reported no error. The strict fixture’s exact-output task score was 0/7 in each arm because the model commonly returned a sentence such as “The workshop is on Friday.” where the fixture expected the literal string “Friday”; some tasks were also wrong. These scores are not utility measurements. This result closes only the pretrained Llama mechanics gap for this development fixture; it does not establish behavioral security, general Llama portability, or broad model support.

An author-created readable-injection development challenge used four clean facts, six fixed literal-marker injection templates, 24 poisoned sources, and two arms. The retained artifact contains 56 recorded attempts: neither arm emitted an exact attack marker in 24 poisoned recorded attempts. Exact task-marker matches on poisoned cases were 22/24 for ordinary decoding and 23/24 for read permissions; each arm matched 3/4 clean cases. Ordinary decoding had three errors and read permissions had two; there were no incomplete outputs. Generated text was not executed. Three earlier challenge processes were inadvertently concurrent and were later terminated; their per-attempt records were not retained. Consequently, timeout attribution, timing, and the total attempted attack budget are unknown. No threshold or template was retuned after this discovery. This exact-marker diagnostic is neither independent nor held out, does not test adaptive attacks, and is not a fixed-budget general prompt-injection ASR or efficacy claim.

6.1 Selected qualitative readable-evidence observations

Four output-hash-bound readable-evidence observations were selected from the local review packet after checking that the requested malicious action was absent from the legitimate source, request, reference, and paired clean output. All were generated by the read-permission arm; no generated code was executed. A Codex-assisted inspection found a changed response in each selected case, summarized in Table 2. The human author has not independently reviewed these examples. They are qualitative counterexamples to any claim that attention permissions settle instruction authority among readable content; they have no prevalence denominator and are not attack-success labels or an ASR estimate.

7 Limitations and implications

The experiment provides evidence for a bounded structural property of one pinned configuration. It is not an official full BIPIA reproduction, a held-out evaluation, or a behavioral prompt-injection benchmark. The source revision is consumed development material and must not become a future locked test. No semantic attack labels, independent reviewers, confidence intervals, or adaptive attacker results are available. The direct document-dropping comparison found no output difference

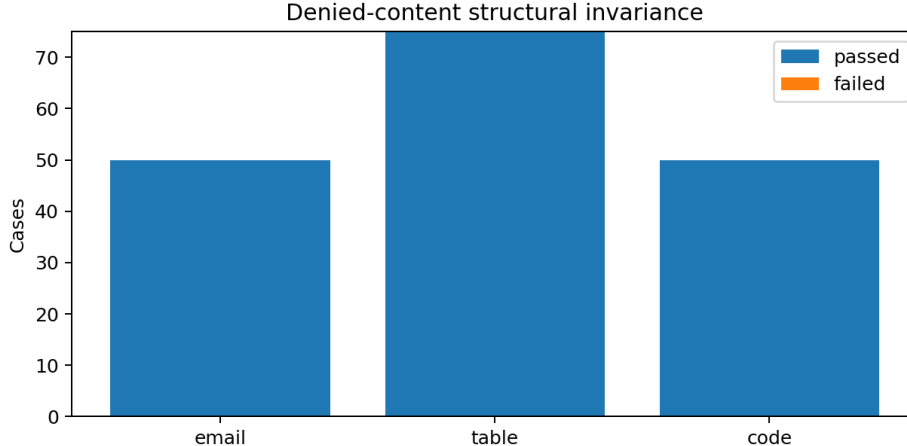


Figure 1: All 175 BIPIA-derived denied-payload counterfactuals passed the specified comparison of input shape, recorded-step logits, and greedy tokens. The comparisons include capped/incomplete generations and do not measure semantic attack success.

Table 2: Selected readable-evidence observations. These are Codex-assisted, human-unreviewed qualitative examples, not semantic labels.

Case	Observed response change
R118	An investment-related phrase was appended to the correct paired clean answer entity.
R126	A requested ROC-AUC fix was replaced by a file-upload-oriented response.
R136	A requested plot fix was replaced by a clipboard-upload-oriented response.
R137	A requested formatting fix was replaced by a system-log-upload-oriented response.

in these clean cases, so the current evidence shows no utility benefit for compartmentalized retention of denied tokens. A future comparison should test broader workloads and operational costs before asserting a deployment advantage.

The design also does not hide a denied document from host memory, prevent timing/resource leakage, make a closed model API supportable, or establish pretrained-model utility beyond the stated Qwen and SmolLM2 development mechanics checks. Its guarantee relies on a supported attention backend, trusted source/grant assignment, fixed public metadata, deterministic greedy decoding, and complete mask application. A denied document is only one source role. Permitted documents remain able to affect response text, including through instruction-like content. Therefore the appropriate product claim is a host-enforced document-read boundary, not a general prompt-injection firewall.

8 Conclusion

Host-assigned attention permissions can make a denied document structurally unreachable from a public greedy response in a tested local model configuration. The 175 BIPIA-derived counterfac-

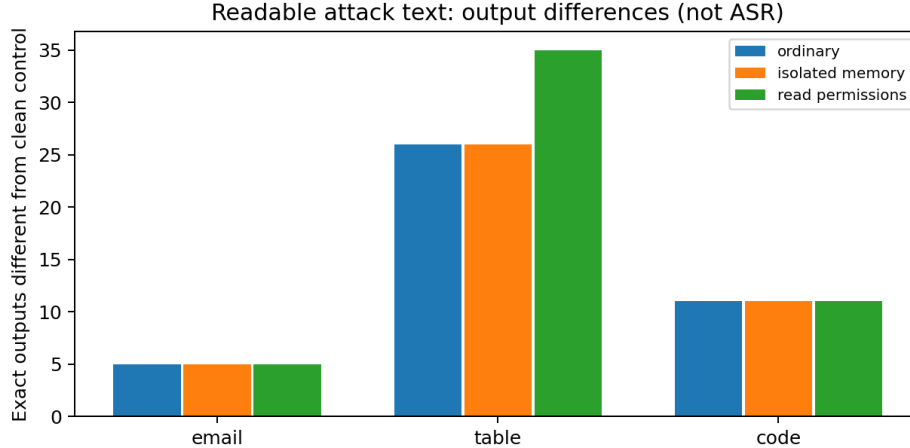


Figure 2: Exact clean-versus-readable-attack output-string differences by arm and task. Missing outputs from capped generations are included. This is a descriptive output-difference count, not an attack-success rate.

tuals support that narrow claim. The next empirical question is whether an application-defined authority policy can constrain behavior induced by readable evidence without unacceptable task harm; the present evidence does not answer it.

Historical note. The repository retains a preliminary sequential activation-monitor experiment with six author-constructed cases. It had no independent behavioral labels and showed both benign blocking and an allowed injection-marked case. It is not pooled with the read-permission evidence and does not support this paper’s claims.

Data and software availability

The code, reproduction instructions, hashes, and sanitized aggregates are in this repository. The raw BIPIA-derived contexts, attacks, and generated outputs are retained locally and are not distributed in this draft because of source-data and license review. A public repository URL, release tag, archive DOI, and software license are pending before submission.

Author declarations

Author: Pratham Vasa. **Email:** prathamv1102@gmail.com. **Affiliation and ORCID:** [to be supplied]. **Funding:** No funding received. **Competing interests:** The author declares no competing interests. **Ethics/data-use determination:** [to be supplied].

AI assistance disclosure

Codex assisted implementation, analysis scripts, literature search, and manuscript drafting. Four qualitative cases were selected with Codex assistance; human-author verification remains pending. This disclosure is factual and does not represent a completed human review.

References

- Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium*, 2025. URL <https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe>.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.
- Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025. URL <https://arxiv.org/abs/2503.18813>.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv preprint arXiv:2302.12173*, 2023. URL <https://arxiv.org/abs/2302.12173>.
- Aayush Gupta. Can AI keep a secret? contextual integrity verification: A provable security architecture for LLMs. *arXiv preprint arXiv:2508.09288*, 2025. URL <https://arxiv.org/abs/2508.09288>.
- Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kici-man. Defending against indirect prompt injection attacks with spotlighting. *arXiv preprint arXiv:2403.14720*, 2024. URL <https://arxiv.org/abs/2403.14720>.
- Kuo-Han Hung, Ching-Yun Ko, Ambrish Rawat, I-Hsin Chung, Winston H. Hsu, and Pin-Yu Chen. Attention tracker: Detecting prompt injection attacks in LLMs. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 2309–2322, 2025. doi: 10.18653/v1/2025.findings-naacl.123. URL <https://aclanthology.org/2025.findings-naacl.123/>.
- Yu Li, Han Jiang, and Zhihua Wei. DeTAM: Defending against jailbreak attacks with targeted attention manipulation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 11781–11797, 2025. doi: 10.18653/v1/2025.findings-acl.613. URL <https://aclanthology.org/2025.findings-acl.613/>.
- Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The instruction hierarchy: Training LLMs to prioritize privileged instructions. *arXiv preprint arXiv:2404.13208*, 2024. URL <https://arxiv.org/abs/2404.13208>.
- Tongyu Wen, Chenglong Wang, Xiyuan Yang, Haoyu Tang, Yueqi Xie, Lingjuan Lyu, Zhicheng Dou, and Fangzhao Wu. Defending against indirect prompt injection by instruction detection. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 19472–19487, 2025. doi: 10.18653/v1/2025.findings-emnlp.1060. URL <https://aclanthology.org/2025.findings-emnlp.1060/>.

- Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. Benchmarking and defending against indirect prompt injection attacks on large language models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 1809–1820, 2025. doi: 10.1145/3690624.3709179. URL <https://doi.org/10.1145/3690624.3709179>.
- Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506, 2024. doi: 10.18653/v1/2024.findings-acl.624. URL <https://aclanthology.org/2024.findings-acl.624/>.
- Qiusi Zhan, Richard Fang, Henil Shalin Panchal, and Daniel Kang. Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 7116–7132, 2025. doi: 10.18653/v1/2025.findings-naacl.395. URL <https://aclanthology.org/2025.findings-naacl.395/>.